

SplitPix Design Document (v2.1)

SplitPix Design Document

Status: v2.1 (language policy added)

Author: Luiz

Target implementation time: 4–6 focused days (day 0 is a walking skeleton; see section 24)

Primary stack: Java 21, Spring Boot, PostgreSQL, JDBC, JUnit 5, Testcontainers, GitHub Actions

1. Executive Summary

SplitPix is a Pix-inspired group-expense application for roommates, trips, dinners, clubs, and other shared purchases.

The application does **not** send real Pix payments or integrate with Brazilian banks. Instead, it helps users:

1. Create an invite-only expense group.
2. Record shared expenses.
3. Assign exact shares to each participant.
4. Compute each participant's net balance.
5. Minimize the number of required repayments.
6. Display the recipient's Pix key and exact amount.
7. Track whether each repayment has been completed.

The project is intentionally designed to demonstrate backend engineering rather than frontend complexity. Its central technical features are:

- Java domain and service-layer design.
- Direct SQL and explicit database transactions.
- Concurrency-safe settlement processing.
- Idempotent write operations.
- Invariant-based JUnit integration tests.
- PostgreSQL-backed tests through Testcontainers.
- Automated verification through GitHub Actions.

The MVP should remain small enough to complete in a few days while still having a real use case and a technically credible architecture.

2. Product Motivation

Existing group-expense applications are useful for tracking debt, but a lightweight Pix-first workflow can reduce friction for Brazilian users.

A typical group-expense flow in Brazil is:

1. One person pays for a shared purchase.
2. Participants calculate how much each person owes.
3. The payer sends a Pix key through WhatsApp.
4. Each participant manually copies the key and amount.
5. Someone keeps track of who has paid.

SplitPix consolidates these steps. It does not replace the banking app; it coordinates the accounting and repayment process around Pix.

Example

Luiz pays **R\$420.00** for dinner for five people. The shares are unequal:

- Luiz: R\$70.00
- Ana: R\$90.00
- Bruno: R\$80.00
- Clara: R\$60.00
- Diego: R\$120.00

SplitPix records the expense, computes each participant's net balance, combines it with previous group expenses, and generates the smallest practical set of payments. Each debtor receives:

- Recipient name.
 - Exact repayment amount.
 - Recipient Pix key.
 - A button to copy payment information.
 - A button to mark the payment as completed.
-

3. Goals

3.1 Product goals

The MVP must allow a small group to:

- Create a group.
- Add participants.
- Store one Pix key per participant.
- Record shared expenses.
- Use exact, unequal expense shares.
- View net balances.
- Generate simplified repayment instructions.
- Mark repayment obligations as settled.
- View a basic transaction history.

3.2 Engineering goals

The implementation should visibly demonstrate:

- Layered Java application design.
- Explicit SQL rather than hidden ORM behavior.
- Atomic multi-table database writes.
- Row-level locking for concurrent settlement requests.
- Idempotency keys for retry-safe write operations.
- Database constraints as a second line of defense.
- Integration tests against real PostgreSQL.
- Tests centered on accounting invariants.
- Continuous integration on every push and pull request.

3.3 Portfolio goals

A reviewer should be able to understand within two minutes that the project includes more than CRUD.

The README should make the following clear:

- SplitPix solves a real coordination problem.
- The system maintains zero-sum group accounting.

- Settlement writes are transactional.
 - Duplicate requests are safe.
 - Concurrent requests cannot settle the same obligation twice.
 - The test suite validates business invariants against PostgreSQL.
-

4. Non-Goals

The MVP will not include:

- Real Pix transfers.
- Open Finance integration.
- Bank account linking.
- QR code payment initiation.
- Identity verification.
- Fraud detection.
- Multi-currency support.
- Interest, fees, or exchange rates.
- Public group discovery.
- Full production authentication.
- Native mobile applications.
- Push notifications.
- Microservices.
- Event streaming.
- Kubernetes.

These features would increase scope without materially improving the main backend demonstration.

5. Target Users

5.1 Primary users

- Roommates sharing rent, groceries, and utilities.
- Friends sharing restaurant or trip expenses.
- Student organizations reimbursing small purchases.
- Sports teams or clubs tracking shared costs.

5.2 User assumptions

For the MVP:

- Groups are private and accessed through an invite token.
 - Participants trust the group creator.
 - Pix transfers occur outside SplitPix.
 - Marking a settlement as complete is a manual confirmation.
 - A participant may be represented without creating a full account.
-

6. Core User Stories

Group management

- As a user, I can create a private group.
- As a user, I can add participants to a group.
- As a user, I can open a group through a private invite URL.

Participant management

- As a group member, I can have a display name.

- As a group member, I can store a Pix key.
- As a group member, I can update my Pix key.

Expenses

- As a group member, I can record that one participant paid for an expense.
- As a group member, I can assign an exact share to each participant.
- As a group member, I cannot save an expense whose shares do not sum to the total.
- As a group member, I can view the group's expense history.

Balances

- As a group member, I can see how much each participant owes or is owed.
- As a group member, I can generate simplified repayment instructions.

Settlements

- As a debtor, I can view the recipient, amount, and Pix key for a repayment.
 - As a user, I can mark a suggested repayment as completed.
 - As a user, retrying the same settlement request does not settle it twice.
 - As a user, two concurrent requests cannot both settle the same obligation.
-

7. Functional Requirements

7.1 Create group

Input:

- Group name.
- Creator display name.
- Optional creator Pix key.

Output:

- Group ID.
- Private invite token.
- Creator participant ID.

7.2 Add participant

Input:

- Group ID.
- Display name.
- Optional Pix key type.
- Optional Pix key value.

Validation:

- Display name must be nonblank.
- Pix key value must be unique within the group.

7.3 Create expense

Input:

- Group ID.
- Description.
- Participant who paid.
- Total amount in centavos.
- List of participant shares.
- Idempotency key.

Validation:

- Total must be positive.
- Every listed participant must belong to the group.
- Every share must be nonnegative.
- Sum of shares must equal the total exactly.
- The payer must belong to the group.
- The idempotency key must be unique for expense creation.

7.4 Calculate balances

For each participant:

$\text{net balance} = \text{total amount paid} - \text{total assigned share}$

Interpretation:

- Positive balance: the group owes this participant.
- Negative balance: this participant owes the group.
- Zero balance: fully settled.

The sum of all participant balances must always equal zero.

7.5 Generate suggested payments

The application should transform participant net balances into repayment instructions.

Input:

- Group balances.

Output:

- Payer participant.
- Recipient participant.
- Amount in centavos.

The algorithm should preserve all net balances while reducing the number of transfers. The MVP does not need to mathematically prove the absolute minimum number of transfers in every possible case; a greedy largest-debtor/largest-creditor algorithm is sufficient.

7.6 Complete settlement

Input:

- Group ID.
- Payer participant ID.
- Recipient participant ID.
- Amount.
- Idempotency key.

Behavior:

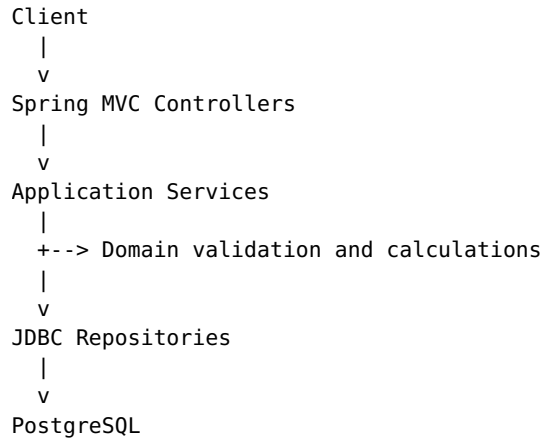
- Validate that the payer owes at least the requested amount.
- Validate that the recipient is owed at least the requested amount.
- Lock the relevant balance or obligation records.

- Insert a settlement record.
- Update accounting state.
- Commit all changes atomically.

A failed settlement must leave no partial state.

8. High-Level Architecture

SplitPix will use a conventional layered monolith.



Layers

Controller layer

Responsibilities:

- Parse HTTP requests.
- Validate basic request structure.
- Call application services.
- Convert domain results into HTTP responses.
- Map exceptions to status codes.

Service layer

Responsibilities:

- Enforce business rules.
- Define transaction boundaries.
- Coordinate repository operations.
- Execute balance calculations.
- Execute settlement simplification.
- Handle idempotency.

Repository layer

Responsibilities:

- Execute SQL.
- Map rows to Java records.
- Perform explicit row locking.
- Avoid business logic.

Database

Responsibilities:

- Store durable state.

- Enforce foreign keys and uniqueness.
 - Enforce basic numerical constraints.
 - Provide ACID transactions.
 - Serialize conflicting settlement operations.
-

9. Proposed Java Package Structure

```
src/main/java/com/luiz/splitpix/
├── SplitPixApplication.java
├── common/
│   ├── DatabaseException.java
│   ├── NotFoundException.java
│   └── GlobalExceptionHandler.java
├── group/
│   ├── Group.java
│   ├── GroupController.java
│   ├── GroupRepository.java
│   ├── GroupService.java
│   └── CreateGroupRequest.java
├── participant/
│   ├── Participant.java
│   ├── ParticipantController.java
│   ├── ParticipantRepository.java
│   ├── ParticipantService.java
│   └── PixKeyType.java
├── expense/
│   ├── Expense.java
│   ├── ExpenseShare.java
│   ├── ExpenseController.java
│   ├── ExpenseRepository.java
│   ├── ExpenseService.java
│   ├── CreateExpenseRequest.java
│   └── ExpenseShareRequest.java
├── balance/
│   ├── ParticipantBalance.java
│   ├── BalanceController.java
│   ├── BalanceService.java
│   └── SuggestedPayment.java
└── settlement/
    ├── Settlement.java
    ├── SettlementController.java
    ├── SettlementRepository.java
    ├── SettlementService.java
    └── CompleteSettlementRequest.java
```

10. Data Model

All money must be represented as integer centavos using Java long and PostgreSQL BIGINT.

Floating-point types such as double must never be used for stored monetary values.

10.1 Groups

```
CREATE TABLE groups (
    id UUID PRIMARY KEY,
    name VARCHAR(120) NOT NULL,
    invite_token VARCHAR(120) NOT NULL UNIQUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

10.2 Participants

```
CREATE TABLE participants (  
  id UUID PRIMARY KEY,  
  group_id UUID NOT NULL REFERENCES groups(id) ON DELETE CASCADE,  
  display_name VARCHAR(100) NOT NULL,  
  pix_key_type VARCHAR(20),  
  pix_key_value VARCHAR(200),  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
  CONSTRAINT valid_pix_key_pair CHECK (  
    (pix_key_type IS NULL AND pix_key_value IS NULL)  
    OR  
    (pix_key_type IS NOT NULL AND pix_key_value IS NOT NULL)  
  ),  
  
  CONSTRAINT unique_pix_key_per_group  
    UNIQUE (group_id, pix_key_value)  
);
```

Supported key types:

- EMAIL
- PHONE
- RANDOM

CPF keys are deliberately excluded from the MVP. CPF is a national identifier; storing it in an application whose access model is an invite token is a privacy liability with no portfolio upside. The exclusion is documented in the README as a design decision.

The application will not attempt full Pix key verification in the MVP. Basic format validation is optional.

10.3 Expenses

```
CREATE TABLE expenses (  
  id UUID PRIMARY KEY,  
  group_id UUID NOT NULL REFERENCES groups(id) ON DELETE CASCADE,  
  paid_by_participant_id UUID NOT NULL REFERENCES  
participants(id),  
  description VARCHAR(200) NOT NULL,  
  total_cents BIGINT NOT NULL,  
  idempotency_key VARCHAR(120) NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
  CONSTRAINT positive_expense_total CHECK (total_cents > 0),  
  CONSTRAINT unique_expense_request UNIQUE (group_id,  
idempotency_key)  
);
```

10.4 Expense shares

```
CREATE TABLE expense_shares (  
  expense_id UUID NOT NULL REFERENCES expenses(id) ON DELETE  
CASCADE,  
  participant_id UUID NOT NULL REFERENCES participants(id),  
  amount_cents BIGINT NOT NULL,  
  
  PRIMARY KEY (expense_id, participant_id),  
  CONSTRAINT nonnegative_share CHECK (amount_cents >= 0)  
);
```

10.5 Settlements

```

CREATE TABLE settlements (
  id UUID PRIMARY KEY,
  group_id UUID NOT NULL REFERENCES groups(id) ON DELETE CASCADE,
  payer_participant_id UUID NOT NULL REFERENCES participants(id),
  recipient_participant_id UUID NOT NULL REFERENCES
participants(id),
  amount_cents BIGINT NOT NULL,
  idempotency_key VARCHAR(120) NOT NULL,
  status VARCHAR(20) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT positive_settlement CHECK (amount_cents > 0),
  CONSTRAINT different_settlement_participants CHECK (
    payer_participant_id <> recipient_participant_id
  ),
  CONSTRAINT valid_settlement_status CHECK (
    status IN ('COMPLETED')
  ),
  CONSTRAINT unique_settlement_request UNIQUE (group_id,
idempotency_key)
);

```

The MVP can store only completed settlements. A PENDING state is unnecessary unless asynchronous payment verification is later added.

10.6 Optional accounting view

Balances can be calculated from expenses, shares, and settlements rather than stored directly.

A participant's balance is:

```

amount paid for expenses
- assigned expense shares
- settlements received
+ settlements paid

```

This derivation avoids duplicated balance state and reduces the chance of inconsistencies.

An SQL query or database view may compute balances dynamically.

11. Balance Calculation

For participant p:

```

expense credits = sum of expenses paid by p
expense debts   = sum of shares assigned to p
payments sent   = sum of completed settlements where p is payer
payments received = sum of completed settlements where p is
recipient

```

```

net balance = expense credits
              - expense debts
              + payments sent
              - payments received

```

Why the settlement signs work:

- A debtor starts with a negative balance.
- Sending money should move the debtor's balance upward toward zero.
- A creditor starts with a positive balance.
- Receiving money should move the creditor's balance downward

toward zero.

Example

Before settlement:

Alice: -5000

Bob: +5000

Alice sends Bob 5000:

Alice: $-5000 + 5000 = 0$

Bob: $+5000 - 5000 = 0$

12. Debt Simplification Algorithm

12.1 Inputs

A list of participant balances whose sum is zero.

12.2 Outputs

A list of suggested payments that would reduce every balance to zero.

12.3 Greedy strategy

1. Put negative balances into a debtor priority queue.
2. Put positive balances into a creditor priority queue.
3. Select the largest debtor and largest creditor.
4. Transfer the smaller of:
 - the debtor's absolute debt;
 - the creditor's remaining credit.
5. Update both balances.
6. Reinsert any participant whose balance is still nonzero.
7. Continue until both queues are empty.

12.4 Pseudocode

```
List<SuggestedPayment> simplify(List<ParticipantBalance> balances) {
    PriorityQueue<ParticipantBalance> debtors = ...;
    PriorityQueue<ParticipantBalance> creditors = ...;
    List<SuggestedPayment> payments = new ArrayList<>();

    while (!debtors.isEmpty() && !creditors.isEmpty()) {
        ParticipantBalance debtor = debtors.poll();
        ParticipantBalance creditor = creditors.poll();

        long amount = Math.min(
            -debtor.balanceCents(),
            creditor.balanceCents()
        );

        payments.add(new SuggestedPayment(
            debtor.participantId(),
            creditor.participantId(),
            amount
        ));

        long newDebtorBalance = debtor.balanceCents() + amount;
        long newCreditorBalance = creditor.balanceCents() - amount;
```

```

        if (newDebtorBalance < 0) {
            debtors.add(debtor.withBalance(newDebtorBalance));
        }

        if (newCreditorBalance > 0) {
            creditors.add(creditor.withBalance(newCreditorBalance));
        }
    }

    return payments;
}

```

12.5 Complexity

For n participants:

$O(n \log n)$

This is more than sufficient for small groups.

12.6 Important limitation

The generated suggestions are derived from the current state. They should not be stored as durable obligations in the MVP because a new expense can invalidate them.

Instead:

- Generate suggestions on demand.
- When a user completes one, submit the payer, recipient, and amount as a settlement.
- Recalculate balances and suggestions after the settlement commits.

This avoids stale suggested-payment records.

13. Transaction Design

13.1 Expense creation transaction

Creating an expense touches multiple tables and must be atomic.

Transaction steps:

1. Check for an existing expense with the same group and idempotency key.
2. Validate that the payer belongs to the group.
3. Validate that every share participant belongs to the group.
4. Validate that share amounts sum exactly to the total.
5. Insert the expense.
6. Insert all expense shares.
7. Commit.

If any insertion fails, the entire transaction rolls back.

13.2 Settlement transaction

Settlement is the main concurrency-sensitive operation.

Transaction steps:

1. Check for an existing settlement with the same idempotency key.
2. Lock the group row (SELECT ... FOR UPDATE).

3. Recalculate both participants' current balances inside the transaction.
4. Verify that the payer's balance is at most -amount.
5. Verify that the recipient's balance is at least amount.
6. Insert the completed settlement.
7. Commit.

13.3 Group-level locking (v2 decision)

```
SELECT id
FROM groups
WHERE id = ?
FOR UPDATE;
```

Both write paths take this lock: settlement completion and expense creation. Reads (balances, suggestions, history) take no lock.

13.4 Why participant-row locking is not enough

Balances are derived from expenses, shares, and settlements. Locking the payer and recipient participant rows serializes competing settlements, but an expense insert changes balances without touching any participant row. A concurrent expense could therefore commit between settlement validation and settlement insertion, breaking invariants 4, 5, and 8.

Locking the group row in every write path serializes all writes within one group, which makes the invariants straightforward to prove and to test. Example, with Alice owing R\$100.00 and two concurrent requests (settle R\$80.00 to Bob, settle R\$80.00 to Carol): the second transaction waits on the group lock, recalculates after the first commits, and fails cleanly because only R\$20.00 remains owed.

For five-person groups the lost concurrency is irrelevant. The tradeoff is documented in the README as a deliberate choice of provable correctness over throughput the application does not need.

13.5 Isolation level

PostgreSQL's default `READ COMMITTED` is sufficient under the group lock: every balance recalculation happens after the lock is acquired, and all validation and insertion occur in the same transaction. `SERIALIZABLE` remains unnecessary.

14. Idempotency

Clients may retry requests after timeouts or network failures. Retrying must not create duplicate expenses or settlements.

14.1 Expense idempotency

Uniqueness:

(group_id, idempotency_key)

If the same request is submitted again:

- Return the already-created expense.
- Do not create another expense or another set of shares.

14.2 Settlement idempotency

Uniqueness:

(group_id, idempotency_key)

If the same settlement request is submitted again:

- Return the existing settlement.
- Do not apply the settlement twice.

14.3 Conflicting reuse

A stronger implementation may store a request hash and reject reuse of the same idempotency key with different request contents.

This is a stretch goal, not an MVP requirement.

15. API Design

Base path:

/api/v1

15.1 Create group

POST /api/v1/groups

Content-Type: application/json

```
{
  "groupName": "Rio Trip",
  "creatorName": "Luiz",
  "pixKeyType": "EMAIL",
  "pixKeyValue": "luiz@example.com"
}
```

Response:

```
{
  "groupId": "uuid",
  "inviteToken": "random-token",
  "creatorParticipantId": "uuid"
}
```

15.2 Get group

GET /api/v1/groups/{groupId}?token={inviteToken}

15.3 Add participant

POST /api/v1/groups/{groupId}/participants

Content-Type: application/json

```
{
  "displayName": "Ana",
  "pixKeyType": "PHONE",
  "pixKeyValue": "+5511999999999"
}
```

15.4 Create expense

POST /api/v1/groups/{groupId}/expenses

Idempotency-Key: expense-2026-07-29-001

Content-Type: application/json

```

{
  "description": "Dinner",
  "paidByParticipantId": "uuid",
  "totalCents": 42000,
  "shares": [
    {
      "participantId": "uuid-1",
      "amountCents": 7000
    },
    {
      "participantId": "uuid-2",
      "amountCents": 9000
    },
    {
      "participantId": "uuid-3",
      "amountCents": 8000
    },
    {
      "participantId": "uuid-4",
      "amountCents": 6000
    },
    {
      "participantId": "uuid-5",
      "amountCents": 12000
    }
  ]
}

```

15.5 Get balances

GET /api/v1/groups/{groupId}/balances

Response:

```

{
  "groupId": "uuid",
  "balances": [
    {
      "participantId": "uuid-1",
      "displayName": "Luiz",
      "balanceCents": 35000
    },
    {
      "participantId": "uuid-2",
      "displayName": "Ana",
      "balanceCents": -9000
    }
  ]
}

```

15.6 Get suggested payments

GET /api/v1/groups/{groupId}/suggested-payments

Response:

```

{
  "payments": [
    {
      "payerParticipantId": "uuid-2",
      "payerName": "Ana",
      "recipientParticipantId": "uuid-1",
      "recipientName": "Luiz",
      "recipientPixKey": "luiz@example.com",
      "amountCents": 9000
    }
  ]
}

```

```
}  
]  
}
```

15.7 Complete settlement

POST /api/v1/groups/{groupId}/settlements
Idempotency-Key: settlement-ana-luiz-001
Content-Type: application/json

```
{  
  "payerParticipantId": "uuid-2",  
  "recipientParticipantId": "uuid-1",  
  "amountCents": 9000  
}
```

15.8 Get history

GET /api/v1/groups/{groupId}/activity

The response may combine expenses and settlements sorted by creation time.

16. Error Handling

Suggested mappings:

Condition	HTTP status
Invalid JSON or missing field	400
Shares do not sum to total	400
Invalid amount	400
Participant not in group	400
Incorrect invite token	403
Group or participant not found	404
Duplicate Pix key	409
Settlement exceeds current debt	409
Idempotency conflict	409
Unexpected database failure	500

Example error response:

```
{  
  "code": "INVALID_EXPENSE_ALLOCATION",  
  "message": "A soma das partes deve ser igual ao valor total da  
despesa."  
}
```

Error code values remain English and stable; user-facing message text is Brazilian Portuguese (see the language policy in section 21). Tests assert on code, never on localized message text.

17. Business Invariants

The test suite should emphasize invariants rather than only endpoint examples.

Invariant 1: Group balances sum to zero

sum of all participant balances = 0

Invariant 2: Expense shares equal expense total

sum of shares for an expense = expense total

Invariant 3: Settlements preserve zero-sum accounting

A settlement changes two participant balances by opposite amounts.

Invariant 4: A participant cannot settle more than currently owed

The resulting accounting state must never cross from debtor to creditor because of an oversized settlement request.

Invariant 5: A recipient cannot receive more than currently owed

The resulting accounting state must never cross from creditor to debtor because of an oversized settlement request.

Invariant 6: Duplicate idempotency keys have one effect

Repeated expense or settlement submissions must create at most one durable record.

Invariant 7: Failed transactions leave no partial state

If expense share insertion fails, no expense remains.

If settlement validation fails, no settlement remains.

Invariant 8: Concurrent settlements cannot over-settle a balance

When two settlement requests compete for the same outstanding debt, only a valid subset may commit.

Invariant 9: Simplified payments preserve original balances

Applying all generated suggested payments to the current balances must reduce every balance to zero.

Invariant 10: Suggested payments contain only positive amounts

No suggested payment may have a zero or negative amount.

18. Testing Strategy

18.1 Unit tests

Use plain JUnit for:

- Balance arithmetic.
- Debt simplification.
- Validation logic.
- Money formatting helpers.

18.2 Integration tests

Use Testcontainers to run PostgreSQL for:

- Expense transaction rollback.
- Settlement transaction rollback.
- Idempotency enforcement.
- Uniqueness constraints.
- Row locking.
- Concurrent settlements.
- Repository SQL correctness.

18.3 API tests

Use Spring Boot integration tests for:

- Successful request/response flows.
- Error mappings.
- Invalid input.
- Group access token behavior.

18.4 Critical concurrency test

Setup:

Alice balance: -10000
 Bob balance: +8000
 Carol balance: +2000

Concurrent requests:

- Alice pays Bob 8000.
- Alice pays Carol 8000.

Expected:

- At most one 8000 settlement succeeds.
- Alice is never over-settled.
- Group balances still sum to zero.
- Exactly one corresponding settlement row exists if only one request is valid.

18.5 Property-style algorithm test

Generate random zero-sum participant balances.

For every generated case:

1. Run the simplification algorithm.
2. Apply the suggested payments in memory.
3. Assert that all final balances are zero.
4. Assert that every suggested amount is positive.
5. Assert that no participant pays and receives in the same individual suggestion.

A full property-testing framework is optional. A randomized JUnit loop is sufficient for the MVP.

19. GitHub Actions CI

```
name: CI

on:
  push:
    branches: ["main"]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Check out repository
        uses: actions/checkout@v4

      - name: Set up Java
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven

      - name: Run verification
        run: mvn --batch-mode clean verify
```

Testcontainers will start PostgreSQL inside the GitHub Actions runner.

The CI pipeline should fail on:

- Compilation errors.
- Unit-test failures.
- Integration-test failures.
- Concurrency-test failures.

Optional later additions:

- Checkstyle.
- SpotBugs.
- JaCoCo coverage report.

These are not required for the initial version.

20. Local Development

Requirements

- Java 21.
- Maven.
- Docker.

Docker Compose

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: splitpix
      POSTGRES_USER: splitpix
      POSTGRES_PASSWORD: splitpix
    ports:
      - "5432:5432"
```

Commands

```
docker compose up -d
mvn spring-boot:run
mvn clean verify
```

21. Frontend Scope

The backend is the priority.

The smallest useful frontend should support:

- Create group.
- Add participant.
- Add expense.
- View balances.
- View suggested payments.
- Copy Pix key.
- Mark payment complete.

Recommended approach for v1:

- REST API only; no frontend in v1.
- A `demo.sh` script that exercises the full flow with `curl`: create group, add participants, add expense, show balances, show suggestions, complete a settlement.
- README with the demo output pasted in, plus an optional terminal GIF.

A Thymeleaf UI is a stretch goal (section 26). It demonstrates nothing the target skill gaps require, and the implementation window belongs to the transactional core.

Language policy

- **Brazilian Portuguese (pt-BR), user-facing:** API error message values, validation messages, payment-instruction text shown to participants, `demo.sh` output labels, and any future UI.
- **English, everything else:** code identifiers, comments, commit messages, README and this document, API paths and JSON field names, logs.
- Machine-readable error code values are English and never change: tests assert on codes, not on localized text.
- Centralize user-facing strings in one place (Spring `MessageSource` with `messages_pt_BR.properties`) instead of scattering literals; adding another locale later becomes a one-line stretch goal.

Rationale: the product's audience is Brazilian friend groups; the engineering audience (recruiters, interviewers, reviewers) reads English. This mirrors standard practice at Brazilian tech companies, and the README should say so in one sentence.

22. Security and Privacy

The MVP is not production-ready, but it should avoid obviously unsafe behavior.

Minimum protections

- Use prepared statements for every SQL query.
- Never log Pix key values in plaintext.

- Treat invite tokens as secrets.
- Generate invite tokens using a cryptographically secure random source.
- Do not expose all groups publicly.
- Validate group membership for every referenced participant.
- Do not store bank credentials.
- Do not claim that payments are verified by a bank.

Pix key privacy

Pix keys may contain sensitive personal identifiers, especially phone numbers (CPF keys are excluded from the MVP for exactly this reason; see 10.2).

For the MVP:

- Show a Pix key only inside its group.
- Mask keys in logs and error messages.
- Consider displaying only a partially masked value except on the payment instruction page.

Authentication limitation

An invite-token model is acceptable for a portfolio MVP but should be documented as a limitation.

A production system would require:

- User authentication.
- Authorization rules.
- Secure participant ownership.
- Key encryption at rest.
- Audit logging.

23. Observability

Keep observability minimal.

Log:

- Request ID.
- Group ID.
- Operation type.
- Success or failure.
- Exception category.

Do not log:

- Full Pix keys.
- Invite tokens.
- Raw request bodies containing sensitive values.

Optional metrics:

- Expenses created.
- Settlements completed.
- Failed settlement validations.
- Duplicate idempotent requests.

Metrics are a stretch goal.

24. Implementation Plan

This schedule assumes no prior Spring Boot experience; the original 2-3 day estimate did not.

Day 0 (half day): Walking skeleton

- Generate the Spring Boot project; add JDBC, PostgreSQL, validation, JUnit, and Testcontainers dependencies.
- One trivial endpoint and one Testcontainers test that talks to real PostgreSQL.
- GitHub Actions green on this skeleton before any feature code exists.
- If Docker or Testcontainers is going to fight, it fights here, not on day 4.

Days 1-2: Core domain

- Schema (schema.sql); group and participant creation.
- Expense transaction with validation, idempotency, and rollback tests.
- Balance derivation as one SQL aggregate; zero-sum invariant test.

Days 3-4: Settlements

- Debt simplification (greedy) plus the randomized property test (18.5).
- Settlement transaction under the group lock; expense path takes the same lock.
- The critical concurrency test (18.4) and the remaining invariant tests.

Day 5: Ship

- README: architecture, API examples, design notes, limitations.
- demo.sh walk-through and sample output.
- Tag v1.0. Stop.

Day 6: Buffer only

- Exists to absorb overruns, not to add features. If unused, the project finishes early.

25. MVP Definition of Done

The MVP is complete when:

- A user can create a group.
- A user can add participants and Pix keys.
- A user can record an expense with exact shares.
- Invalid allocations are rejected.
- Group balances are calculated correctly.
- Suggested repayments are generated.
- A repayment can be marked complete.
- Settlement writes are atomic.
- Repeated idempotency keys have one effect.
- Concurrent settlements cannot over-settle a participant.
- PostgreSQL-backed tests pass locally.
- GitHub Actions passes on every push.
- The README clearly states that SplitPix does not send real Pix payments.
- User-facing messages are in Portuguese, per the language policy, and the README documents the split.

26. Stretch Goals

Only consider these after the MVP is complete:

- Thymeleaf web UI.
 - Equal-split and percentage-split helpers.
 - Expense editing with an audit trail.
 - Participant removal rules.
 - Shareable WhatsApp repayment message.
 - Pix QR code payload generation without bank integration.
 - User accounts and authentication.
 - Email notifications.
 - Export to CSV.
 - Deployment to Render, Railway, or Fly.io.
 - Request hashing for strict idempotency semantics.
 - Database migration tooling with Flyway.
 - OpenAPI documentation.
-

27. Known Tradeoffs

Derived balances versus stored balances

Decision: Derive balances from expenses, shares, and settlements.

Advantages:

- One source of truth.
- Easier invariant reasoning.
- Lower risk of balance drift.

Disadvantages:

- More expensive queries.
- Settlement validation requires recalculation.

For small groups, correctness is more important than query optimization.

Suggestions generated on demand versus stored obligations

Decision: Generate suggestions on demand.

Advantages:

- Suggestions never become permanently stale.
- No need to update obligation rows after every expense.

Disadvantages:

- Suggested payment IDs do not exist.
- The client must submit payer, recipient, and amount when settling.

This is acceptable for the MVP.

JDBC versus ORM

Decision: Use JDBC or Spring JdbcTemplate.

Advantages:

- SQL is visible in the repository.
- Transactional behavior is easier to explain.
- Row locking is explicit.

Disadvantages:

- More mapping code.
- More boilerplate.

The visibility of SQL is a portfolio advantage for this project.

Invite tokens versus authentication

Decision: Use invite tokens initially.

Advantages:

- Much smaller scope.
- Immediately usable by a friend group.

Disadvantages:

- Weak identity model.
- Anyone with the token can modify the group.

This limitation must be documented clearly.

28. Resolved Decisions (v2)

1. **Balance derivation:** one SQL aggregate query (four legs via FILTER or UNION ALL). Visible SQL is the portfolio point, and a single query keeps the settlement transaction simple.
 2. **Lock target:** the group row, in both write paths. See 13.3.
 3. **Is participant-row locking sufficient?** No. See 13.4.
 4. **Group-level lock tradeoff:** accepted deliberately and documented in the README.
 5. **Idempotency key transport:** HTTP header (Idempotency-Key), Stripe-style, stored in the table.
 6. **CPF keys:** excluded from the MVP for privacy reasons; EMAIL, PHONE, RANDOM only.
 7. **Settlement reversal:** none. Append-only; corrections are opposite-direction settlements. Accounting rows are never edited.
 8. **Expense editing:** excluded, same append-only rule.
 9. **Simplification algorithm:** greedy. Exact transfer-count minimization is NP-hard; greedy guarantees at most $n-1$ transfers, which is optimal enough for real groups and one sentence of good interview material.
 10. **Transaction mechanism:** Spring @Transactional with JdbcTemplate (know the proxy self-invocation caveat; it is a stock interview question).
-

29. Concurrency Simplification (adopted)

The group-level lock originally proposed here is the v2 design and has been folded into section 13. It applies to both settlements and expense creation. Finer-grained locking remains a possible later version, with the analysis in 13.4 as the bar it must clear.

30. Recommended Initial Technical Decisions

To minimize implementation risk:

- Use Java 21 records for immutable DTOs and domain results.
 - Use Spring Boot and JdbcTemplate.
 - Use Spring @Transactional for transaction boundaries.
 - Use PostgreSQL.
 - Use Flyway if setup is quick; otherwise use `schema.sql` initially.
 - Derive balances from immutable expense and settlement records.
 - Treat expenses and settlements as append-only.
 - Generate suggested payments on demand.
 - Lock the group row during settlement and expense creation.
 - Use Testcontainers for every repository integration test.
 - Externalize user-facing strings (Spring MessageSource, pt-BR); tests assert on error codes, never on message text.
 - Ship v1 as a REST API; Thymeleaf is a stretch goal.
-

31. README Summary

SplitPix is a Pix-inspired group-expense application built with Java and PostgreSQL. It helps groups record shared expenses, calculate net balances, minimize repayments, and coordinate settlement through Pix keys. The application does not transfer real money; it provides payment instructions and maintains a transactional accounting record. Expense and settlement operations are idempotent and atomic, while PostgreSQL locking prevents concurrent requests from creating inconsistent balances. The test suite uses JUnit and Testcontainers to verify zero-sum accounting, rollback behavior, debt simplification, and concurrency safety. User-facing text is Brazilian Portuguese by design; code, tests, and documentation are in English.

32. Résumé Description

SplitPix — Java, Spring Boot, PostgreSQL, JUnit, GitHub Actions

- Built a group-expense platform inspired by Pix, Brazil's instant-payment system, that computes net obligations and generates simplified repayment instructions.
 - Implemented atomic SQL transactions, idempotent expense and settlement requests, and PostgreSQL locking to prevent duplicate or conflicting repayments.
 - Developed JUnit and Testcontainers invariant tests for zero-sum balances, allocation conservation, rollback behavior, and concurrent settlement safety; automated verification through GitHub Actions.
-

33. Final Scope Rule

The project succeeds if the transactional accounting core is correct, tested, and clearly documented.

Do not delay completion to add:

- A polished frontend.
- Real payment processing.

- Authentication infrastructure.
- Sophisticated deployment architecture.
- Unnecessary framework abstractions.

A small, complete, concurrency-tested system is stronger than a large unfinished one.